

Java + Spring Boot Task-by-Task Learning Guide (Employee Management REST API)

Step-by-step from zero to CRUD + Database + Validation + Clean Errors

Date: February 07, 2026

This guide teaches Java + Spring Boot the way you will use it on the job by building one real backend project. You will build an Employee Management REST API with professional structure: **Controller -> Service -> Repository -> Database**.

How to use this guide: Do tasks in order. Do not jump. After each task, run the app and test using Postman (or curl). If you get an error, fix one thing at a time.

Outcome: a Spring Boot backend that supports POST, GET, PUT, PATCH (optional), DELETE, validation, clean error responses, and persistence using H2 + JPA.

Contents

Task 0 - Setup and prerequisites

- 1 Task 1 - Create and run Spring Boot app
- 2 Task 2 - Build your first REST API (GET /hello)
- 3 Task 3 - Create Employee model (POJO) + getters/setters
- 4 Task 4 - Create employee (POST /employees)
- 5 Task 5 - Store in memory + read all (GET /employees)
- 6 Task 6 - Create Service layer + dependency injection
- 7 Task 7 - Add database (H2 + JPA) + Repository
- 8 Task 8 - Validation + Global exception handling
- 9 Task 9 - Update (PUT) and Delete (DELETE)
- 10 Task 10 - PATCH (partial update) - optional
- 11 Appendix - Postman test pack + common errors

Task 0 - Setup and prerequisites

Goal: install tools and confirm you can run Java + Maven + Spring Boot.

Install: Java 17+ (Java 21 is OK), IntelliJ IDEA, Postman.

Create a workspace folder: e.g., **springboot-learning**.

Step-by-step

- Open Terminal (or PowerShell). Run: **java -version**. Confirm Java prints version 17+.
- Run: **mvn -v** (or use the project Maven Wrapper later). Confirm Maven prints a version.
- Open IntelliJ -> Settings -> Build Tools -> Maven. Ensure it points to your installed Maven or wrapper.
- Install Postman. Confirm you can create a request tab.

Done when: Java + Maven commands work and you can open IntelliJ without errors.

Task 1 - Create and run Spring Boot app

Goal: generate a Spring Boot app and start the server.

What you learn: project structure, main class, embedded Tomcat.

Step-by-step

- Go to start.spring.io.
- Project: Maven, Language: Java, Spring Boot: latest stable, Packaging: Jar, Java: 17+.
- Group: com.kcorp, Artifact: LearnJava (or any name).
- Dependencies: select **Spring Web**.
- Generate project and unzip.
- Open the folder in IntelliJ (File -> Open).
- Wait for dependencies to download (bottom status bar).
- Run the main class annotated with **@SpringBootApplication**.

Done when

- Console shows: Tomcat started on port 8080.
- No red errors on startup.

Task 2 - Build your first REST API (GET /hello)

Goal: create a controller endpoint that returns a string.

Concepts: @RestController, @GetMapping, URL path mapping.

Step-by-step

- Create package: **com.kcorp.controllers**.
- Create class: **HelloController**.
- Add: **@RestController** on the class.
- Create method returning String and add: **@GetMapping("/hello")**.
- Run the app and open: <http://localhost:8080/hello>

Done when: browser shows 'Hello World' (or your text).

Task 3 - Create Employee model (POJO) + getters/setters

Goal: create a plain Java object for employee data.

Concepts: private fields, getters/setters, packages.

Step-by-step

- Create package: **com.kcorp.model**.
- Create class: **Employee**.
- Add private fields: Long id, String name, String email, String department, double salary.
- Use IntelliJ: Right-click inside class -> Generate -> Getter and Setter -> select fields.
- Keep the class simple: no controller annotations.

Task 4 - Create employee (POST /employees)

Goal: accept JSON and return the created object.

Concepts: @PostMapping, @RequestBody.

Step-by-step

- Create controller class (example name: Employees or EmployeeController).
- Add class annotation: @RestController.
- Add POST method mapped to /employees.
- Parameter must be: @RequestBody Employee employee.
- Return the Employee object.

Postman test

POST http://localhost:8080/employees
Content-Type: application/json

```
{  
  "name": "Krishna Varma",  
  "email": "krishna.varma@kcorp.com",  
  "department": "Engineering",  
  "salary": 95000  
}
```

Done when: you see the same JSON returned (including salary as 95000.0).

Task 5 - Store in memory + read all (GET /employees)

Goal: temporarily store data while the app is running (no database yet).

Concepts: List, ArrayList, class-level state.

Step-by-step

- In controller (or service later), create: `List employees = new ArrayList<>();` at class level.
- In POST: `employees.add(employee);` then return employee.
- Add GET /employees that returns the list.
- Test: POST twice, then GET returns both employees.
- Restart app: list becomes empty again (expected).

Task 6 - Service layer + dependency injection

Goal: move logic out of controller into service (job standard).

Concepts: @Service, constructor injection, clean controller.

Step-by-step

- Create package: com.kcorp.service.
- Create class: EmployeeService and annotate with @Service.
- Move employees List and add/get methods into EmployeeService.
- In controller, remove List and inject EmployeeService using constructor.
- Controller methods call employeeService.addEmployee(...) and employeeService.getEmployees().

Task 7 - Add database with H2 + JPA

Goal: store employees in database (persistence).

Concepts: Entity, Repository, JPA, H2 Console.

Step-by-step

- Add dependencies: spring-boot-starter-data-jpa and h2.
- Convert Employee to @Entity with @Id and @GeneratedValue; id type = Long.
- Create EmployeeRepository extends JpaRepository in com.kcorp.repository.
- Update EmployeeService to inject EmployeeRepository.
- Replace in-memory list: addEmployee -> repository.save, getEmployees -> repository.findAll.
- In application.properties enable H2 console and set JDBC URL to jdbc:h2:mem:testdb.
- Restart app, open: <http://localhost:8080/h2-console>, set JDBC URL to jdbc:h2:mem:testdb, connect.
- POST an employee, then in console run: `SELECT * FROM EMPLOYEE;`

Task 8 - Validation + clean errors

Goal: reject bad input and return friendly error JSON.

Concepts: @NotBlank, @Email, @Positive, @Valid, @RestControllerAdvice.

Step-by-step

- Add validation annotations on Employee fields: @NotBlank(name, department), @Email + @NotBlank(email), @Positive(salary).
- In controller POST and PUT, add @Valid before @RequestBody.
- Create GlobalExceptionHandler class annotated with @RestControllerAdvice.
- Handle MethodArgumentNotValidException and return a map of field -> error message.
- Test with invalid JSON and confirm 400 response + field errors.

Invalid POST example:

```
{
  "name": "",
  "email": "abc",
  "department": "",
  "salary": -10
}
```

Task 9 - PUT update + DELETE

Goal: complete CRUD operations.

Concepts: update existing entity, not-found exception, delete by id.

Step-by-step

- Create ResourceNotFoundException in com.kcorp.exception.
- PUT /employees/{id}: service must find by id, update fields on existing, save existing.
- If id not found: throw ResourceNotFoundException (do not return null).
- Add exception handler for ResourceNotFoundException to return HTTP 404.
- DELETE /employees/{id}: if exists deleteById, else throw ResourceNotFoundException.

Task 10 - PATCH partial update (optional)

Goal: update only the fields provided in request body.

Beginner approach: accept Map and set fields present.

Warning: PATCH is partial; do not require all fields.

PATCH example:

```
PATCH http://localhost:8080/employees/1  
{ "salary": 120000 }
```

Appendix - Postman test pack + common errors

POST /employees: create employee (no id).

GET /employees: list all.

PUT /employees/{id}: full update.

DELETE /employees/{id}: delete.

Validation: invalid body returns 400 with field map.

Common errors

- 405 Method Not Allowed: using wrong HTTP method (browser sends GET). Use Postman.
- H2 'C:/Users/.../test not found': H2 console JDBC URL is wrong. Use jdbc:h2:mem:testdb.
- Controller 404: controller package not under main package; keep packages under com.kcorp.
- NullPointerException: service field set to null; use constructor injection.
- Validation not triggered: missing @Valid on controller parameter.
- PUT saves wrong record: using body id; always use path id to find existing.